FP32 and FP16 transformer optimizations

Controlled comparison of LoRA, KV caching, and attention optimization

Sports QA • GPT-2 • Prepared October 27, 2026

This report first audits Owen's two FP16 PDFs, then presents a fresh matched rerun. It separates training cost, generation speed, and answer accuracy so that "performance" has an explicit meaning.

Matched FA1-derived comparison: same algorithm in both precisions

Version correction: The installed PyTorch 2.14 FlashAttention backend defaults to FA2; the prior custom FP32 tiled kernel is not established as the original FA1 implementation. Those two backends cannot be described as FA1-to-FA1. The installed registry and version evidence are saved in `ATTENTION_VERSION_AUDIT.json`. See PyTorch's attention documentation.

To hold algorithm generation fixed, this added experiment uses one FA1-derived Triton prototype for both FP32 and FP16. It follows the original paper's key/KV tiles outer, query tiles inner loop; normalized output and row statistics are written between key tiles, and backward recomputes probabilities in the same loop order. Both policies use the same 16-by-32 tiles, one program per batch/head, and FP32 output/statistic accumulators. FP32 dot products request IEEE arithmetic; FP16 Q/K/V and dot operands use half precision. Reference: FlashAttention, Algorithms 1 and 4.

This is a matched FA1-derived algorithm prototype, not the original optimized FA1 CUDA implementation. It tests this code under two precisions; its speed must not be presented as production FA1 performance or as a comparison between official FA1 releases. The original production implementation does not supply the project's custom IEEE-FP32 baseline.

Hardware: NVIDIA A30, job 2421762. Same selected checkpoints from the equal-budget LR sweep; B1/P896, 64 generated tokens, three warmups, seven measured repeats with shuffled modes. The eager baselines are remeasured in this allocation. No samples are pooled with the deployment-backend experiment. Each precision uses its own validation-selected trained checkpoint; within each precision the backend/cache switches hold weights fixed. Cross-precision selected-model differences therefore include training-policy effects and are not a pure weight-fixed cast experiment.

FP32 — same FA1-derived code

Configuration	Request ms	Baseline speedup	Peak MiB	Primary answer	Answer + scores
Baseline: full / eager / no cache	2300.1 ± 5.6	1.00x	797.8	19.2%	19.2%
FA1-derived alone	4670.7 ± 2.4	0.49x	756.8	19.2%	19.2%
KV cache alone	417.5 ± 0.9	5.51x	871.7	19.2%	19.2%
LoRA alone	2526.7 ± 2.1	0.91x	799.7	80.8%	79.2%
All three	609.6 ± 0.7	3.77x	852.7	80.8%	79.2%
Remove FA1-derived attention	622.0 ± 1.6	3.70x	874.0	80.8%	79.2%
Remove KV cache	4870.3 ± 4.0	0.47x	759.0	80.8%	79.2%
Remove LoRA: full adaptation	399.5 ± 0.5	5.76x	850.4	19.2%	19.2%

Changes from all three: remove attention: time +2.0%, primary accuracy +0.0 pp; remove cache: time +699.0%, primary accuracy +0.0 pp; replace LoRA with full fine-tuning: time -34.5%, primary accuracy -61.7 pp. Positive time means slower.

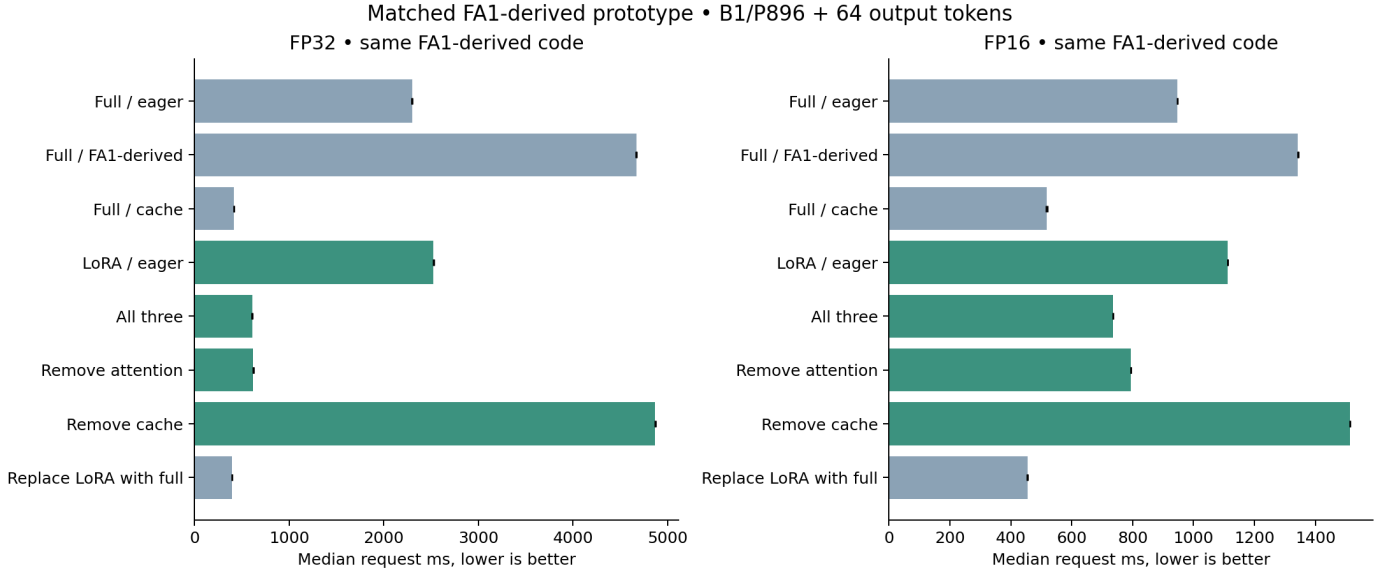
Fastest latency among these eight rows: Remove LoRA: full adaptation at 399.5 ms. All three is 52.6% slower than that minimum; compare answer accuracy before choosing it.

FP16 — same FA1-derived code

Configuration	Request ms	Baseline speedup	Peak MiB	Primary answer	Answer + scores
Baseline: full / eager / no cache	946.8 ± 0.5	1.00x	775.1	34.2%	33.3%
FA1-derived alone	1342.8 ± 0.5	0.71x	775.1	34.2%	33.3%
KV cache alone	518.2 ± 1.7	1.83x	857.2	34.2%	33.3%
LoRA alone	1111.5 ± 0.4	0.85x	777.4	82.5%	82.5%
All three	735.9 ± 0.7	1.29x	857.0	82.5%	82.5%
Remove FA1-derived attention	793.9 ± 0.7	1.19x	858.2	82.5%	82.5%
Remove KV cache	1513.8 ± 0.6	0.63x	777.4	82.5%	82.5%
Remove LoRA: full adaptation	455.1 ± 1.0	2.08x	857.4	34.2%	33.3%

Changes from all three: remove attention: time +7.9%, primary accuracy +0.0 pp; remove cache: time +105.7%, primary accuracy +0.0 pp; replace LoRA with full fine-tuning: time -38.2%, primary accuracy -48.3 pp. Positive time means slower.

Fastest latency among these eight rows: Remove LoRA: full adaptation at 455.1 ms. All three is 61.7% slower than that minimum; compare answer accuracy before choosing it.



The tables answer the removal question with attention generation held fixed. Cache/backend switches preserve the selected checkpoint; full fine-tuning substitutes an equally budgeted task adaptation and therefore changes learned weights. ± is timing sample SD, not seed uncertainty.

Matched FA1-derived training cost

Adaptation	Precision	Attention	Step ms	Peak MiB
full	FP32	Eager	244.2	5996
full	FP32	FA1-derived	307.8	5167
full	FP16	Eager	98.7	5896
full	FP16	FA1-derived	93.1	4627
lora	FP32	Eager	189.3	4745
lora	FP32	FA1-derived	255.2	3916
lora	FP16	Eager	81.8	4655
lora	FP16	FA1-derived	76.3	3382

These are reset-step comparisons at batch 2 / length 896, with the same original starting weights, batch, zero AdamW moments and disabled dropout. All measured updates must be successful and finite. This is step cost, not a training-quality claim; KV cache is inference-only. Full 128/512/896 results and raw samples are preserved under `fa1_matched/`.

Validation: 8 FP32/FP16 forward-and-gradient cases passed against float64 causal attention, covering odd lengths and cached offsets. Float32 tolerance is 3e-5 absolute / 3e-4 relative; FP16 is 0.012 / 0.04. The recorded errors and compiled forward PTX are retained; FP32 PTX excludes MMA and TF32 instructions. Model QA runs check finite logits and preserve before/after weight hashes. Limitations: a new untuned prototype, one training seed, one allocation, no attention dropout or higher derivatives.

Largest recorded forward / gradient absolute error: FP32 6.14e-07 / 2.70e-06; FP16 1.01e-03 / 1.60e-03.

Deployment supplement: PyTorch FA2 versus custom tiled FP32

The following multi-workload results describe the available deployment backends. They compare custom tiled FP32 with PyTorch FA2 under FP16, and are not an FA1-to-FA1 or pure-precision backend comparison. Read the matched FA1-derived section above for generation-controlled evidence.

1. Deployment findings

At batch 1 with an 896-token prompt and 64 output tokens, FP16 LoRA + PyTorch FA2 + KV cache takes **707.0 ms**, versus **947.6 ms** for the task-trained full-fine-tuning/eager/no-cache baseline (1.34x faster). These models have the same adaptation budget and equal tuning budgets; the selected learning rate can differ.

FP32: removing attention optimization changes request time by +12.2%; removing cache changes it by +326.4%; replacing LoRA with equally budgeted full fine-tuning changes it by -38.1%. Primary answer accuracy is 80.8% with LoRA, 19.2% with full fine-tuning, and 25.8% with no task adaptation. Positive time changes mean slower.

FP16: removing attention optimization changes request time by +13.2%; removing cache changes it by +1.5%; replacing LoRA with equally budgeted full fine-tuning changes it by -40.7%. Primary answer accuracy is 82.5% with LoRA, 34.2% with full fine-tuning, and 21.7% with no task adaptation. Positive time changes mean slower.

LoRA's training savings and its inference behavior are different effects. Unmerged adapters add work at inference. Removing adaptation may hurt accuracy while making generation faster; merging adapters retains their learned update and is measured separately below. KV caching applies to autoregressive inference, so a "three-in-one training speedup" is not defined here.

Owen's LoRA parameter/memory arithmetic is credible, but its unequal training budgets do not establish a speed or quality win. His cache PDF contains conflicting labels, hardware, agreement claims, and mismatched provenance. Use the fresh results below for comparisons.

2. Audit of Owen's FP16 reports

The reviewed PDFs and supporting files are preserved in `owen_evidence/`, with repository commit and SHA-256 hashes in `manifest.json`. The raw run files named `report.json` and `report_1.json` by the LoRA PDF could not be identified as

the reported FP16 pair in the checkout, so its reported values are distinguished from independently reproduced measurements.

Report / claim	Assessment	Resolution
LoRA: 589,824 trainable vs 163,037,184 full	Consistent: 276.4x fewer trainable values; rank-4 adapters add 2.25 MiB at FP32 storage	Confirm trainable counts and unchanged frozen weights in fresh runs
LoRA: 3,197.8 to 1,278.8 MiB	60.0% saving is arithmetically plausible; FP16 autocast retains FP32 parameters and optimizer state	Measure both methods at identical length/batch/budget; do not transfer this percentage across workloads
LoRA: 39.6 vs 57.7 ms/step; 12/12 answers	300 vs 10 steps, cold-start imbalance, tiny different QA task; not a controlled speed/quality comparison	Fresh 1,200-step runs and two LRs for both methods and precisions on the existing sports benchmark
KV PDF: described as FP16	Repeatedly labeled FP32; 3.34 MiB cache at B1/P32 corresponds to 95 x 36 KiB, i.e. FP16 K/V, rather than the stated 72 KiB/token	Record actual cache byte counts and explicitly define AMP storage
KV PDF: hardware and provenance	Page 2 says A30; page 5 says A100/job 2408911. Saved JSON says A30/job 2347442 and has different timings	Rerun on one allocation with environment metadata
KV PDF: all outputs preserved	Table says 99/120 pretrained and 96/120 LoRA; prose says 360/360. Displayed batch-4 cache speedups are below 1 despite favorable scaling claim	Report actual answer agreement and accuracy; allow measured slowdowns

The cache-byte inconsistency is a diagnosis of the PDF's labels, not proof of how its unpreserved run executed. The copied cache JSON is explicitly FP32 and internally consistent with its older FP32 experiment; it does not verify the newer PDF's displayed numbers.

3. What is held constant

Control	Fresh protocol
Hardware	NVIDIA A30; gpu-4-21.int.turing.wpi.edu; main job 2421704; PyTorch 2.14.0+cu130; CUDA 13.0
Data	Existing synthetic sports QA: 1,440 training questions, 120 validation, 120 test; unseen test team names; exact input hashes retained
Training budget	1,200 updates, batch 8, padded length 128; seed 42; identical sample-order hash across all eight candidates
Optimizer and adapters	AdamW, weight decay 0, gradient norm clipped at 1; training dropout 0.1; LoRA rank 4 and alpha 8 on QKV, attention-output and both MLP projections in all 12 blocks
Tuning	Two LRs (0.001 and 0.0001) per method per precision; choose minimum validation NLL at 300/600/900/1200; freeze choices before any test generation
Precision	FP32 parameter storage in both policies; FP16 means CUDA autocast with FP16 projections/attention and GradScaler initialized at 128; TF32 off
Inference	Same prompt IDs, greedy decoding, exactly 64 new tokens; B1/P128, B1/P512, B1/P896, B4/P128; last-position output head in every mode
Timing	Three warmups, seven synchronized measured repeats; shuffle backend/cache modes each repeat; loading, compilation, tokenization, transfers and cleanup excluded
Quality	120 held-out questions; 40 generated tokens, score through EOS; both primary-answer and stricter answer-plus-score-field accuracy
Memory	Peak PyTorch allocated bytes; cache bytes already included; reserved memory is distinct and is not added
Baseline	Main factorial baseline uses task-trained full fine-tuning + eager attention + cache off. Untouched pretrained baseline shown separately

FP32 attention optimization is the custom Triton tiled implementation with IEEE FP32 dot products and first-order backward. FP16 attention optimization is forced PyTorch FlashAttention-2 (FA2). These are distinct implementations; there is no official

FP32 FlashAttention result to relabel. Backend gains are measured against eager at the same precision.

Fresh FP32-versus-FP16 selected-model comparisons include any effect of training under each precision. The untouched pretrained model provides a fixed-weight precision comparison. All inference uses AMP rather than native-half parameter storage; the earlier native-FP16 supplement remains a separate deployment-policy experiment.

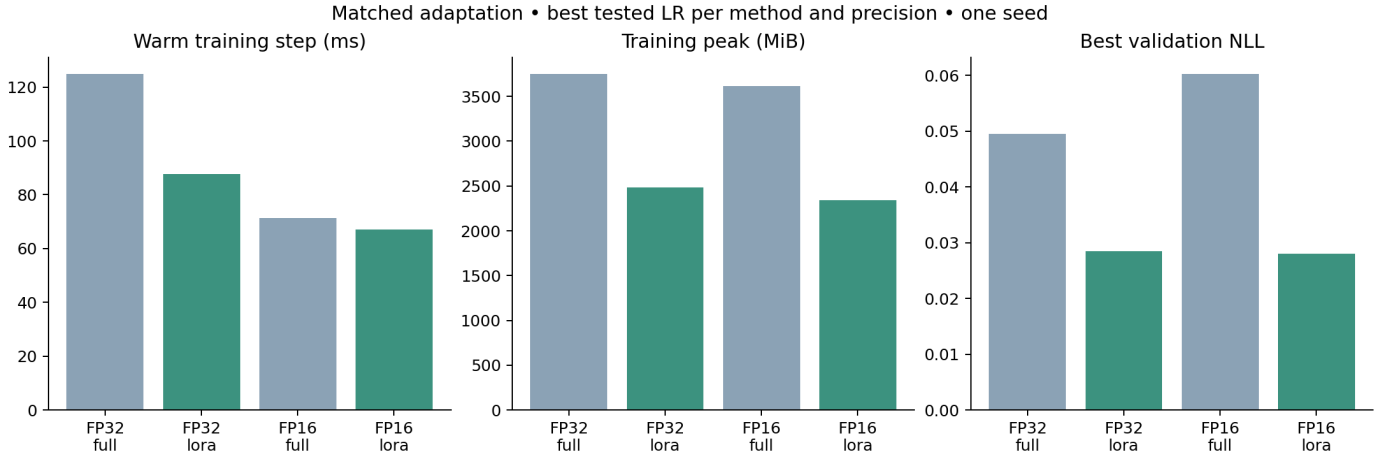
4. Best tested versus suboptimal learning rates

Precision	Method	LR	Selection	Step	Val NLL (lower)	Warm ms/step (lower)	Peak MiB (lower)
FP32	full	0.001	Suboptimal tested	900	0.15015	123.8	3747
FP32	full	0.0001	Best tested	1200	0.04948	124.9	3751
FP32	lora	0.001	Best tested	1200	0.02853	87.7	2483
FP32	lora	0.0001	Suboptimal tested	900	0.07750	87.7	2481
FP16	full	0.001	Suboptimal tested	600	0.15097	71.2	3614
FP16	full	0.0001	Best tested	600	0.06026	71.4	3614
FP16	lora	0.001	Best tested	1200	0.02801	67.0	2341
FP16	lora	0.0001	Suboptimal tested	900	0.07592	66.9	2340

Held-out answer quality for every rate, measured only after all validation choices were frozen (eager attention, cache on):

Precision	Method	LR	Primary answer	Answer + scores
FP32	full	0.001	15.0%	12.5%
FP32	full	0.0001	19.2%	19.2%
FP32	lora	0.001	80.8%	79.2%
FP32	lora	0.0001	50.8%	50.8%
FP16	full	0.001	14.2%	10.0%
FP16	full	0.0001	34.2%	33.3%
FP16	lora	0.001	82.5%	82.5%
FP16	lora	0.0001	49.2%	49.2%

These test scores are descriptive and do not change the validation-based selection. "Suboptimal" is defined by validation NLL, so a candidate can still do better on an individual test metric.



"Best tested" means lowest validation NLL among these two learning rates and four checkpoint opportunities. It is not a global optimum over rank, alpha, batch size, optimizer, or training budget. The unselected rate is called suboptimal only within this tested set. Validation NLL compares fit under equal budgets; warm step time and peak memory show cost. No test score was used to pick a rate.

Fresh parameter counts match the expected design: full fine-tuning trains 163,037,184 scalar values; LoRA trains 589,824 adapter values out of 163,627,008 total (0.3605%), while preserving the frozen base.

All eight runs completed 1,200 real optimizer updates with finite clipped gradients. No overflow-skipped update was counted as a fast step. Warm step medians exclude the first ten steps. These are one-seed training trajectories, not estimates of seed uncertainty.

5. Each optimization, all three, and each removal

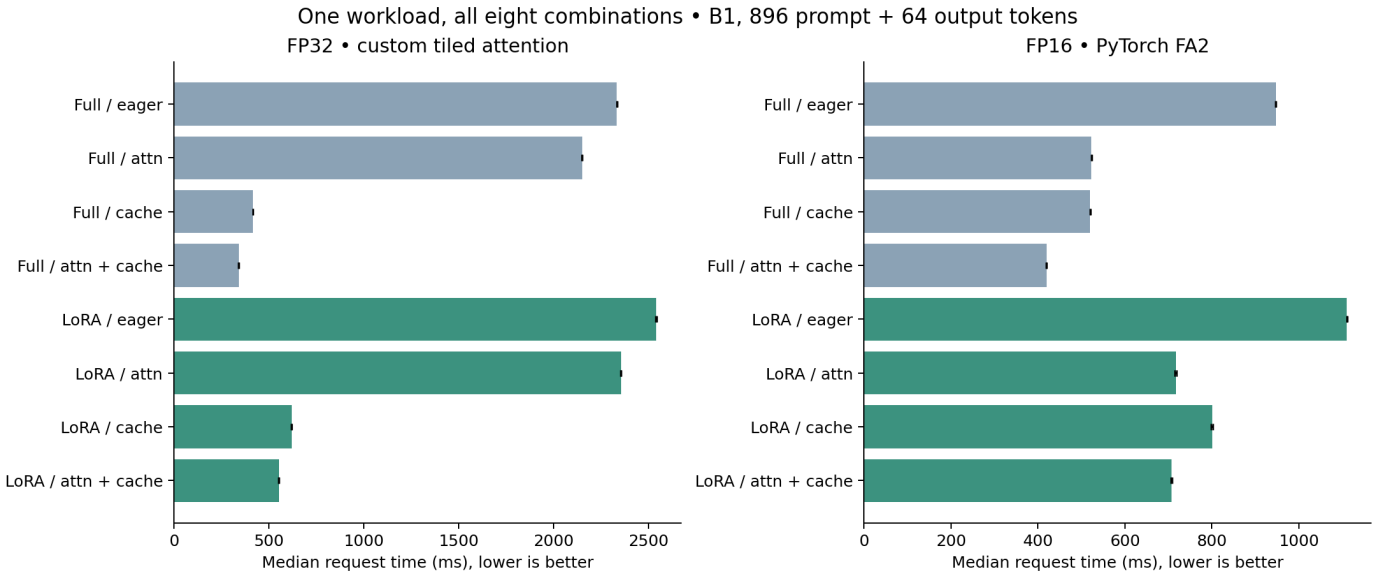
Main workload: batch 1, prompt 896, 64 new tokens. Each row uses a validation-selected task-trained checkpoint. "Attention" means tiled FP32 or FP16 PyTorch FA2. Cache changes only inference execution.

FP32

Configuration	Request ms (lower)	Baseline speedup (higher)	Peak MiB (lower)	Primary (higher)	Answer + scores (higher)
Baseline: full adaptation	2333.6 ± 1.2	1.00x	795.9	19.2%	19.2%
Attention alone	2149.9 ± 1.1	1.09x	755.7	19.2%	19.2%
KV cache alone	415.5 ± 0.6	5.62x	870.4	19.2%	19.2%
LoRA alone	2541.0 ± 3.5	0.92x	798.1	80.8%	79.2%
All three	552.2 ± 0.9	4.23x	851.3	80.8%	79.2%
Remove attention	619.4 ± 2.0	3.77x	872.6	80.8%	79.2%
Remove KV cache	2354.6 ± 2.1	0.99x	757.9	80.8%	79.2%
Remove LoRA: full adaptation	341.5 ± 1.7	6.83x	849.1	19.2%	19.2%

FP16

Configuration	Request ms (lower)	Baseline speedup (higher)	Peak MiB (lower)	Primary (higher)	Answer + scores (higher)
Baseline: full adaptation	947.6 ± 0.7	1.00x	773.2	34.2%	33.3%
Attention alone	523.0 ± 0.5	1.81x	773.2	34.2%	33.3%
KV cache alone	520.4 ± 0.6	1.82x	854.1	34.2%	33.3%
LoRA alone	1110.9 ± 0.6	0.85x	775.5	82.5%	82.5%
All three	707.0 ± 1.6	1.34x	857.6	82.5%	82.5%
Remove attention	800.6 ± 2.4	1.18x	857.7	82.5%	82.5%
Remove KV cache	717.4 ± 2.0	1.32x	775.5	82.5%	82.5%
Remove LoRA: full adaptation	419.4 ± 0.5	2.26x	852.9	34.2%	33.3%



± is sample standard deviation of seven timing repetitions, around the reported median; it is not a confidence interval or cross-seed uncertainty. The same quality score is repeated across synthetic timing shapes because it comes from the separate held-out QA evaluation. Latency does not include task training.

6. Does removing LoRA hurt more than removing the others?

There are three different LoRA removals. The first is the primary fair adaptation comparison; the second removes task training entirely; the third retains the learned update and removes its runtime branches.

Precision	Change from all three	Time change	Primary change	Peak MiB change
FP32	Remove attention	+12.2%	+0.0 pp	+21.3
FP32	Remove cache	+326.4%	+0.0 pp	-93.4
FP32	Replace LoRA with full fine-tuning	-38.1%	-61.7 pp	-2.2
FP32	Remove task adaptation	-38.1%	-55.0 pp	-2.2
FP32	Merge LoRA into base weights	-37.5%	+0.0 pp	-10.5
FP16	Remove attention	+13.2%	+0.0 pp	+0.1
FP16	Remove cache	+1.5%	+0.0 pp	-82.1
FP16	Replace LoRA with full fine-tuning	-40.7%	-48.3 pp	-4.8
FP16	Remove task adaptation	-41.0%	-60.8 pp	-4.8
FP16	Merge LoRA into base weights	-40.8%	+0.0 pp	-10.1

Positive time means slower; negative accuracy means worse; pp means percentage points. Removing cache or switching attention preserves the same learned checkpoint. Full fine-tuning and no adaptation change model weights, so their accuracy differences are adaptation comparisons rather than isolated kernel effects.

The merge supplement ran in a separate allocation (NVIDIA A30, job 2421709). It merges W = W + (alpha/rank)BA in FP32 and replaces all 48 adapter modules with ordinary linear layers. A same-input FP32 logit check passes at combined 2e-4 absolute/relative tolerance. Its timing differences versus the main run are descriptive, because allocation/order effects are not paired. Quality is measured again rather than assumed bitwise identical.

7. Training: LoRA and attention, with matched baselines

KV caching is inactive during these full-sequence supervised training steps. The table measures one successful forward/backward/AdamW update after resetting the original pretrained weights and optimizer moments before every trial. Batch 2, length 896; dropout disabled equally for all rows because the custom FP32 attention kernel has no attention-dropout support. This isolates step cost, rather than convergence.

Trainable method	Precision	Attention	Step ms (lower)	Peak MiB (lower)
full	FP32	Eager	243.0	5997
full	FP32	Tiled	264.4	5168
full	FP16	Eager	99.2	5900
full	FP16	FA2	72.5	4562
lora	FP32	Eager	188.5	4744
lora	FP32	Tiled	211.1	3915
lora	FP16	Eager	82.2	4651
lora	FP16	FA2	55.7	3321

The full 128/512/896-length results, seven raw samples per variant, gradient finiteness, loss scale, and successful-update assertions are retained in `confirmation-full-training.json` and `confirmation-lora-training.json`. Training master weights and AdamW tensors remain FP32 in both policies. LoRA reduces the trainable set; it does not remove the base model.

8. Where the benefits change with workload

Precision	Batch / prompt	Baseline ms	All-three ms	Combined gain	Cache gain in all-three	Attention gain in all-three
FP32	1 / 128	445.9	542.8	0.82x	1.02x	1.12x
FP32	1 / 512	1239.3	540.4	2.29x	2.36x	1.13x
FP32	1 / 896	2333.6	552.2	4.23x	4.26x	1.12x
FP32	4 / 128	1250.4	543.3	2.30x	2.48x	1.15x
FP16	1 / 128	514.2	701.7	0.73x	1.00x	1.14x
FP16	1 / 512	534.3	714.3	0.75x	1.01x	1.13x
FP16	1 / 896	947.6	707.0	1.34x	1.01x	1.13x
FP16	4 / 128	561.9	756.0	0.74x	0.97x	1.15x

Ratios below 1 mean slowdown. Gains must be read from the measured combination; multiplying isolated speedups from different workloads is invalid. Prompt length and batch size are workload controls here, not accuracy hyperparameters to call "optimal." All measured prompt-plus-output lengths stay below the 1,024-token context limit. At saturation the cache implementation rebuilds the cropped window to preserve learned-position behavior, limiting its benefit.

9. Fixed-weight precision check and answer agreement

Precision	Attention	Cache	Request ms	Peak MiB	Cache MiB	Primary
FP32	fp32_eager	Off	2323.8	795.9	0.0	25.8%
FP32	fp32_eager	On	415.9	870.4	67.4	25.8%
FP32	fp32_tiled	Off	2146.7	755.7	0.0	25.8%
FP32	fp32_tiled	On	341.8	849.1	67.4	25.8%
FP16	fp16_eager	Off	949.6	773.2	0.0	24.2%
FP16	fp16_eager	On	515.8	854.1	33.7	21.7%
FP16	fp16_flash	Off	523.2	773.2	0.0	21.7%
FP16	fp16_flash	On	416.8	852.9	33.7	21.7%

These pretrained weights are untouched in both precisions, so eager FP32 versus eager AMP FP16 is a fixed-weight policy comparison. Its low sports-task score is a reference, not the main trained baseline. The logical cache size is 2 x layers x batch x cached tokens x hidden width x element size: 72 KiB/token in FP32 and 36 KiB/token for FP16 K/V. After 64 predictions the cache contains prompt + 63 tokens.

Precision	Checkpoint	Attention	Cache off/on identical through EOS
FP32	pretrained	fp32_eager	120/120
FP32	pretrained	fp32_tiled	120/120
FP32	full	fp32_eager	120/120
FP32	full	fp32_tiled	120/120
FP32	lora	fp32_eager	120/120
FP32	lora	fp32_tiled	120/120
FP16	pretrained	fp16_eager	101/120
FP16	pretrained	fp16_flash	118/120
FP16	full	fp16_eager	120/120
FP16	full	fp16_flash	120/120
FP16	lora	fp16_eager	120/120
FP16	lora	fp16_flash	120/120

Floating-point reduction, casting, and different input shapes can change greedy tokens even with valid cache logic. Report answer scores and exact agreement separately. Every answer, token sequence, and component score is preserved in the per-model quality JSON files.

10. Confidence, limits, and reproduction

Fresh GPU validation: **18 tests, 0 failures, 0 errors, 0 skips**, job 2421712. Coverage includes KV caching, LoRA frozen weights, FlashAttention, and custom FP32 tiled forward/backward. Before/after inference weight hashes match for all six main checkpoints; all frozen LoRA base values match after training. All eight candidates use the same sample-order hash.

This is one A30 allocation and one training seed. Backend/cache timing modes are randomized within each model; different architectures and training candidates run sequentially. Timing SD describes repetition noise, not seed uncertainty. Small timing changes relative to this spread should not be treated as a reliable win or loss without more allocations. The synthetic sports scorer is format-sensitive, and three questions share each match; it does not establish broad language-model quality. Historical three-seed FP32 sports results remain in the original matched study and are not pooled with this fresh one-seed run. No counterfactual test or native-FP16 retraining was added in this rerun.

The fresh rerun confirms controlled performance and quality on the existing sports task; it does not exactly reproduce Owen's separate 12-question fictional-team experiment. His raw FP16 run provenance was insufficient for that exact rerun. Deployment all-three compatibility here means an FP16-trained LoRA checkpoint executing with forced PyTorch FA2 and cache, with finite logits, weight preservation and recorded QA outputs; KV cache is not used during training.

Reproduce from this directory: `sbatch run_gpu.sh`; after it finishes, `sbatch run_merge_gpu.sh`, `sbatch run_candidates_gpu.sh`, and `sbatch run_fa1_gpu.sh`; independently run `sbatch validate_gpu.sh`. The main runner refuses to overwrite `confirmation.json`; use a new output directory with copied scripts for a new measurement. Build with `python build_report.py` in an environment containing NumPy, Matplotlib, Markdown and ReportLab. The builder currently locates optional report libraries under `/tmp/optimization-report-libs`; install those packages in the active environment for portability. GPU runtime is configured by the existing `~/MQP/Torch/setup.sh`.

Artifacts: `OPTIMIZATION_COMPARISON.pdf`, standalone `OPTIMIZATION_COMPARISON.html`, this Markdown report, `FA1_MATCHED_METRICS.csv` with 16 matched-algorithm cells, `METRICS.csv` with all 96 deployment inference cells, `ALL_OUTPUTS.csv` with every main QA answer, `confirmation.json`, `merged-confirmation.json`, validation logs, raw training traces, selected checkpoints, source snapshots, input hashes, and the preserved Owen evidence. Earlier reports are left intact.